SUTRA · TRAFFIC INTELLIGENCE

Feature roadmap

Everything in *Features to add in SUTRA* (15 October 2026), turned into an ordered path. Fourteen phases, each one ending in something that runs, is tested, and can be shown to somebody.

15 October 2026 Sarvesh Kumar Yadav Source: featuressutra.pdf
Record: docs/project-record.html

How this is ordered

The source document lists a headline feature and a backlog ranked High / Medium / Low by effort against payoff. This roadmap keeps those priorities but sequences by **dependency**, because two of them cannot be built in the order they were written:

> **A live bug outranks every feature.** Tapping an emergency destination (hospital, fire, police) in Delhi drew a 2,000 km route to Bengaluru. The three chips were hardcoded coordinates with invented names ("Metro Police HQ"). On the emergency path, a wrong destination is worse than a missing one, so this is Phase 0.

> **The surroundings view cannot use Realtime.** The source document's first build step says to subscribe to nearby vehicles. Row-level security lets a driver read only their *own* position row, and Supabase Realtime only delivers rows the subscriber may select — so a driver's subscription returns their own vehicle and nothing else. It needs a server-side function instead, which is why Phase 2 is larger than the document assumes, and why the simulator moves up to Phase 3.

PHASE	WHAT IT DELIVERS	PRIORITY
0	Emergency destinations that are actually nearby	DONE · 15 OCT
1	Shared geometry: bearing, projection, relative position	DONE · 15 OCT
2	Live surroundings view — radar + map toggle	DONE · 15 OCT
3	Vehicle simulator	DONE · 15 OCT
4	Real-device demo run	HIGH
5	Push to a closed app (FCM)	BUILT · 15 OCT PHONE TEST PENDING
6	Trip replay + violations view	DONE · 16 OCT
7	Collision / near-miss warning	DONE · 16 OCT
8	Richer violation types	DONE · 16 OCT
9	Geofencing + event engine	DONE · 16 OCT
10	PostGIS spatial queries	DEFERRED — SEE TRIGGER
11	Per-road speed limits	DONE · 16 OCT
12	Signal-adapter interface	DONE · 16 OCT
13	Offline buffering	DONE · 16 OCT

The phases

PHASE 0 **Fix the emergency destinations** BUG DONE · 15 OCT

`PRESETS` in `mobile/app/dashboard.tsx` were static Bengaluru points, and both routing and corridor planning fell back to `here ?? BANGALORE` — so with no GPS fix the app routed from a guessed city centre rather than refusing.

- New Edge Function `supabase/functions/nearby/`: Overpass `nwr(around:R,…)[amenity=hospital|police|fire_station]`, widened through 5 / 15 / 40 km until something is found, sorted by distance. Reuses the mirror list, `Accept: application/json` header and real user agent that the corridor function already needs, because the public Overpass servers answer 406 otherwise.
- Nominatim fallback with `bounded=1` — without it the viewbox is only a soft preference, which is the same bug more quietly.
- Chips become live lookups around the driver, labelled with the real name and distance ("Fortis Hospital · 2.4 km"), disabled until a location is known, cached once per service.
- The `BANGALORE` fallback is deleted from both routing and corridor planning. Without a fix they now refuse.

> **Found on deployment, and it changed the shape of the fix.** The function was deployed and called for a real Delhi coordinate. Overpass answered `406`, its mirror `500`, and Nominatim `403` — while the identical query from an ordinary network returned `200` with the right stations. The public servers refuse the data centre, not the query. The app therefore also retries straight from the phone, exactly as it already does for the corridor's junctions, and the two paths widen the search identically.

Shipped: 15 handler tests, 9 app tests for the lookup and its fallbacks, 5 dashboard tests for the refusal paths. Totals are now 174 database and handler tests, 113 app tests.

PHASE 1 **Shared geometry** DONE · 15 OCT

Small and boring, and it unblocks Phases 2, 7 and 8. `heading` is stored in two tables and **read by nothing**; no bearing function exists anywhere in the project.

- `mobile/src/geo.ts`: `distanceM`, `bearing`, `project(from, bearing, metres)`, `toRelative(ego, other)`. The planar formula shared with the database's `approx_distance_m` lives in one place — the file exists as of Phase 0 and already holds `distanceM`.
- GPS heading is `0` whenever the phone is stationary, so a derived bearing from the previous fix is needed above walking pace.
- Shipped as `mobile/src/geo.ts`: `distanceM`, `bearing`, `project`, `normalizeDeg`, `toRelative` and `headingFrom`, with 16 tests pinning the compass convention and the inverse relationship between bearing and projection.
- The matching SQL `bearing_deg` was *not* added. Every function in the public schema is a REST endpoint, and nothing server-side needs a bearing until Phase 7 — so it is written there instead of shipping an unused one now.

PHASE 2 **Live surroundings view** DONE · 15 OCT

The driver at the centre, other SUTRA vehicles around them, moving smoothly. Radar canvas first, with a toggle to the map-centred version.

Server

A `nearby_vehicles(vehicle_id, radius_m)` security-definer function, polled at the telemetry cadence, because Realtime cannot do this (see above). Each rule gets a test:

- the caller must own the vehicle they are asking as;
- radius clamped, result capped, rows older than ~15 s excluded;
- **no identity leaves the server** — an opaque per-call reference, kinematics and vehicle type only. Never a plate or a driver name. This is what keeps a connected-awareness feature defensible under the DPDP Act, and it is worth saying out loud in a pitch;
- a `warned` flag, so vehicles currently told to give way can be flashed.

Client

- `surroundings.ts` — poll loop and dead reckoning. This is what makes it look live: positions arrive every 1–3 s, so between updates each vehicle is carried forward along its own heading and speed, and a ghost stops moving after about six seconds without news.
- `Radar.tsx` — ego fixed centre, pointing up; range rings at 100 / 200 / 400 m; others placed by *relative* bearing. Built from plain views and Reanimated, which the project already has — no new native module, because a new one cannot be tested before it is installed on a phone.

> **Stated in the view itself,** the way the corridor's SIM badge is: only SUTRA vehicles appear. No pedestrians, no unconnected traffic, no lane detail — that needs sensors on the vehicle. An empty radar means nobody nearby is running SUTRA, not that the road is clear. This paragraph is on the screen, not only in this document.

Shipped: migration `20261015000012_nearby_vehicles.sql` (applied live and rehearsed in a rolled-back transaction against the real project), `surroundings.ts`, `Radar.tsx` and an *Around me* pane with the Radar / Map toggle. Polling follows the telemetry cadence — every second for an authorized emergency vehicle, every three otherwise — and only while the pane is open. 14 database tests, 23 app tests for the module and the component, 5 more for the pane.

PHASE 3 **Vehicle simulator** DONE · 15 OCT

Ranked Medium in the source document, but Phase 2 cannot be *demonstrated* without it: the radar needs other vehicles and there is one phone. Nothing like it exists in the repository today.

A script that signs in, registers vehicles plated `SIM 0001` onward and drives each along a real OSRM route, upserting positions through the same client the app uses. The triggers, violations, corridor warnings and the radar therefore exercise production code — none of them can tell the difference — and row-level security applies exactly as it would to a driver. `--speeder` makes one exceed the limit, which is the fastest way to see an automatic violation alert land on a real phone.

Speed wanders rather than holding constant, and dips toward a crawl now and then: a fleet moving at exactly 38 km/h is the thing that makes a demo look like a demo. Ctrl-C deletes the vehicles; `--cleanup` clears leftovers.

Shipped and run live (16 Oct): a two-minute run with five cars produced a speeding alert, a real collision warning between two of them at a junction, and a school-zone message, with no false harsh-braking or wrong-way flags. Running it for real found two bugs, both fixed: clean-up silently deleted nothing (vehicles have no delete policy, by design), so removal now goes through an operator-only function for simulated vehicles; and hold-ups braked hard enough to count as harsh braking, so speed now changes no faster than an ordinary driver. Instructions and the recorded run: `tools/README.md`.

PHASE 4 **Real-device demo run** HIGH

Not code. Install the build, drive a route with the simulator running alongside, record it, and write what was actually observed into the project record. Until this happens, "not verified on a device" stands — and it is the first thing a jury or a traffic authority will ask about.

PHASE 5 **Push to a closed app** BUILT · 15 OCT PHONE TEST PENDING

Every notification today is *local*: an alert arrives over the open websocket and the app posts it itself. A closed app hears nothing, and emergency alerts expire in minutes, so they are already stale by the next launch.

- A device-token table with own-rows-only access, and registration through the existing EAS project id.
- A `notify` function posting to Expo's push service with the existing alert channel and high priority, so the device wakes.
- Fired from the alert trigger through `pg_net` — available on this project but not installed, so the extension needs the same guard the scheduler already uses, or every schema test breaks.

> **Needs you:** FCM V1 credentials — a Firebase project, its service-account key uploaded to EAS, and a new build. It cannot be done from the repository.

What shipped

- `notify` takes only an alert id. Recipients and text come from `claim_alert_push`, which stamps `pushed_at` first — so a replayed call sends nothing, and the function is safe behind the public key.
- Addressed alerts go to the phone in that vehicle, not every phone the owner has. Expired alerts go nowhere, and each push lives only as long as its alert: a late give-way warning is wrong, not merely late.
- Push is sent even to open phones, because Realtime has died silently before. The two arrivals collapse into one: both use the identifier `sutra-alert-<id>`, which the library source confirms is the Android tag for both a local notification and a push, so the second replaces the first; and the app silences a push it has already sounded.
- An alert can never fail to save because push could not be requested — the trigger stands aside on any error, and is tested to.
- Signing out withdraws the phone's token, so a shared phone stops receiving the last driver's alerts.

Verified live: an alert insert travelled trigger → `pg_net` → `notify` → claim and returned `200`; a replay changed nothing; an unauthenticated call was refused with `401`. The security advisor then caught `pg_net` installed in `public`, which was moved to `extensions` and the chain re-verified. The last step — a real notification on a real phone — is blocked only on Firebase.

Tests: 19 database tests, 20 for the function, 22 for the app's registration and de-duplication. Push receipts (Expo's delivery confirmation, checked minutes later) are not yet read.

PHASE 6

Trip replay and violations

DONE · 16 OCT

> **Built from moving points only.** A parked phone keeps streaming and GPS wanders a few metres per fix, so summing every point turned an hour parked into kilometres "driven" — a flaw the console's Insights already had. `vehicle_trips()` drops points below walking pace, which also splits trips at stops for free. On the live project it found one genuine 247 m trip and correctly ignored hours of stationary streaming from the same day.

Shipped: a trip list in the console's Insights that opens a replay — play, pause, scrub, 1×/10×/60× — with a moving marker, live speed (red over the limit) and a pin wherever a rule was broken; a *Speeding map* heatmap weighted by how far over the limit; and a *Trips and history* screen in the app. Points are fetched a page at a time, so a long trip is not cut off at PostgREST's 1000-row default. 11 database tests, 6 for the replay arithmetic, 12 in the app.

- A replay scrubber over a vehicle's stored track in the console, with violation pins and a density view.
- A driver-facing history screen: recent trips, distance, top speed, own violations. Row-level security already permits exactly this.
- ~~A bug to fix on the way: the console subscribes to violation inserts, but that table is not in the Realtime publication.~~ Fixed on 15 October, pulled forward because the console was not updating live at all — see below.

PHASE 7

Collision / near-miss warning

DONE · 16 OCT

> **Scoped to what phone GPS can support.** GPS is good to 5–10 m and lanes are 3.5 m apart, so a plain "they will pass within 12 m" test fires on every oncoming car and every overtake — those genuinely pass within 4 m. Only paths crossing at 30–150° are considered: the junction case, where drivers often cannot see each other. Rear-end and lane-change warnings need lane-level positioning and are not attempted. Braking drivers, parked vehicles and stale positions are skipped; each pair is warned once per encounter, and each driver is told which side the danger is on.

Shipped and live: rehearsed with the two real vehicles — the ambulance told "from your left", the other car "from your right". 18 tests, most of them cases that must stay silent, plus one proving a failure in the check can never stop a phone's tracking (it caught that settings read in a PL/pgSQL `DECLARE` escape the exception handler).

A trigger beside the corridor one: for vehicles within a few hundred metres, project both forward and compute time to closest approach; warn both when that is short and close. Dedupe and cool down the way corridor warnings already do — the failure mode here is a phone beeping continuously in slow traffic, so the cooldown and a minimum closing speed *are* the feature, not a detail.

PHASE 8 **Richer violation types** DONE · 16 OCT

Shipped and live. Adding a second kind exposed two assumptions the speeding code had made: it took "the open violation" without asking its kind, and the alert trigger said "Overspeeding" for everything. Both fixed first.

- **Harsh braking** (0.4 g, a setting): recorded, never announced — beeping at someone in the second after an emergency stop distracts them at the worst moment.
- **Wrong way** uses OpenStreetMap roads loaded by `tools/import-roads.mjs` (from a laptop: Overpass refuses Supabase's runtime). Three consecutive fixes against a one-way road, above 10 km/h, and no road nearby the vehicle may legally be on — on a dual carriageway the opposite side is within GPS error. The driver gets an URGENT alert naming the road. Around the test phones the importer found 1,774 roads and 9,298 segments.

Roads loaded (16 Oct): 2,337 roads around the test phones. The first real run met an outage of the main Overpass server (502 on its own status page), so the importer now tiles large areas, tries four servers, retries refused tiles and saves as it goes. Further areas need the importer run once each.

- **Harsh braking** — deceleration against the previous stored position, with the threshold as data rather than code.
- **Wrong direction** — honestly scoped. Without a one-way road layer this is only reliable *against a corridor route*. Doing it for any road needs OpenStreetMap one-way data, so it waits for Phase 11.

PHASE 9 **Geofencing and an event engine** DONE · 16 OCT

Shipped and live. One `events` table with a fixed set of kinds that every check now writes into: zone enter/exit, overspeed, harsh brake, wrong way, collision warning, corridor start/end. The console's log reads from it live. Operators draw zones on the map (school, hospital, restricted, depot); a zone can carry a message sent to the driver on entry — "School zone: slow down, children crossing" — which arrives by push like any alert. Leaving takes two fixes outside, so GPS jitter on a boundary does not flap, and re-entering within two minutes is the same visit. Rehearsed live with a test school zone. 17 tests.

Named zones, and one `events` table with a real taxonomy — zone enter and exit, overspeed, harsh brake, conflict, corridor start and end — which the existing triggers emit into instead of each inventing its own shape. The console's event log becomes a view over one table rather than strings assembled in the browser.

PHASE 10 **PostGIS spatial queries** DEFERRED · 16 OCT

Decision: not now, and here is when. Measured on the live project with every check running against the imported roads, one position update takes about **6 ms** in the database, against one update every 1–3 seconds per vehicle. At today's scale (a handful of vehicles, about 13,000 road segments) PostGIS would make nothing faster that anyone could notice, and it carries a real cost: the test harness cannot load it, so every spatial query would need two versions and the PostGIS one could only be tested on a separate branch database.

Why it will be needed later. Two lookups grow with scale. The wrong-way and speed-limit checks search `road_segments` on every position, and its index narrows only by latitude, so the cost grows with the area imported: fine for one district, tens of milliseconds per check for a whole city's few hundred thousand segments. And the radar and collision checks scan every live position with no location index at all: fine at a few vehicles, noticeable in the hundreds. A GiST spatial index fixes both.

Trigger to do it: roads imported for a whole city, or a few hundred vehicles active at once — whichever comes first. Before either, it can be proven with a measurement: load a city's worth of synthetic roads and vehicles in a rolled-back transaction and time the per-position cost. That number, not the choice of database, is the answer to "does it scale?".

The original plan, kept for when the trigger is reached:

The trap: PostGIS is available on the project but not installed, and the test harness runs Postgres compiled to WebAssembly, which *cannot load it*. The moment a migration says `create extension postgis` unguarded, every schema test breaks.

1. Guard the extension the way the scheduler is already guarded.
2. Add a geography column and spatial index, keep the planar function as the fallback path, and prefer the index when it exists.
3. Verify on a database branch, since it cannot be covered locally — and say so in the README, rather than letting a test count imply coverage that does not exist.

The payoff is real: stored positions grow by thousands of rows per vehicle-hour, and this is the direct answer to "does it scale?".

PHASE 11 **Per-road speed limits** DONE · 16 OCT

Shipped and live. The limit at a vehicle's position now comes from, in order: a zone an operator gave a limit (a school zone at 25); the limit posted on the road it is on; a default for that kind of road, in a table left deliberately empty because Indian limits are set by state notification and vary by vehicle class; and the global 80 as before. "The road it is on" uses the same direction test as wrong-way, so a service lane is not held to the highway beside it. Rehearsed on an imported road: *"Overspeeding: 60 km/h on Crossings Republik Road, where the limit is 40 km/h"* — a speed the old single limit would never have flagged. The console's zone editor takes a limit. 15 tests.

A speed-zone table seeded from OpenStreetMap `maxspeed`, replacing one global 80 km/h, falling back to the global value where no zone is known. Cleanest once the spatial work of Phase 10 is in place.

PHASE 12 **Signal-adapter interface** DONE · 16 OCT

Shipped and live, switched off. Every junction change on a corridor is logged as the request SUTRA would send a city's controller — `PREEMPT` with an arrival estimate, `HOLD_GREEN`, `RELEASE` — shown on the console beside the corridor. The written contract is `docs/signal-adapter.md`: signed HTTPS requests, the city may refuse any of them, and every request carries a maximum hold the controller must enforce on its own. A dispatch function (deployed, inert until an authority sets an endpoint and switches the mode) and a reference city-side adapter are tested against each other over real HTTP, signature included. Rehearsed live: the real ambulance's junction got PREEMPT (45 s away), HOLD_GREEN, and RELEASE when the corridor ended early.

> **Found while building it:** nothing ever ended a corridor whose phone died. It stayed active indefinitely, its junctions were never released, and the nightly prune never removed it. A job now ends any corridor past its hour or whose vehicle has been silent for two minutes — a junction must not stay held because a battery ran out.

The single most useful item for an authority pitch, because it turns "simulated" into "here is where your controller plugs in".

- A written interface: request pre-emption for a junction at a time, confirm, release.
- A function stub with the real shape and a simulation driver that does what the corridor trigger does today.
- A command log recording every instruction that *would* have been sent. A reviewer can then see exactly what SUTRA asks of a city — and that SUTRA never assumed control of anything.

PHASE 13 **Offline buffering** DONE · 16 OCT

Shipped: positions that fail for want of a network are kept on the phone (surviving a restart) and sent once a live position gets through again — as *history only*, through a checked function, never as the live position. Replaying old fixes as live ones would have moved the vehicle backwards on the map and set off every safety check on stale data. Trips and replay include the stretch without signal. Speeding is deliberately not judged on late, phone-timestamped positions. Future timestamps, anything over 12 hours old, duplicates and malformed points are skipped; a refusal that would recur is not kept. Rehearsed live: five positions into history, the live position untouched. 9 database tests, 16 app tests.

Queue positions on the phone when the network drops, flush oldest-first on reconnect, capped so it cannot grow without bound. Because the live position is a single upserted row, the queue mainly serves history continuity — a naive flush would replay stale positions as though they were current.

Fixed along the way

15 OCT **The console stopped updating until reloaded** BUG FIXED

Reported from real use: the operator console had to be refreshed again and again. Three separate causes, none of them the one first suspected — the publication and the Realtime service both turned out to be healthy, and a probe showed an unpublished table does *not* break a channel.

- **Nothing ever rejoined.** The subscribe callback set the indicator to "disconnected" and stopped there. A slept laptop, a tunnel or a changed network ended the stream permanently, and reloading the page was the only cure.
- **Realtime kept an expired token.** The sign-in token lasts an hour. Nothing passed the refreshed one on, so Realtime went on authorizing rows against a dead token while the socket stayed open — the console looked connected and silently showed nothing.
- **Violations were never published.** That handler had never fired once.

Now: automatic rejoin with backoff, the refreshed token handed to Realtime, a re-read of live positions after every reconnect and when the tab regains focus, and — the part that matters most — a **15-second reconciliation regardless of Realtime**. Realtime is the fast path, not the only one, so a stream that silently dies costs seconds of staleness rather than a reload.

16 OCT **Email confirmation links opened localhost** BUG FIXED

Found when a second person signed up on their own phone: the confirmation email's link opened `localhost:3000`. Neither the app nor the console told Supabase where to send people, so it used its default. The address was in fact already confirmed — Supabase verifies before redirecting, and that account signed in a minute later — but a dead page reads as failure. Both sign-up paths now point at a real page, `/confirmed.html`, which says what to do next, explains expired or reused links, and strips the sign-in token from the address. The project's Site URL and redirect allow-list were set in the Supabase dashboard the same day, and a new sign-up confirmed through the page.

16 OCT **Reporting rate follows speed** CHANGED

Normal vehicles reported every 3 s regardless of speed — at 60 km/h, 50 m between fixes, which blunted collision, zone and wrong-way checks. The database's ~6 ms per update is not the limit; a phone's GPS, at about one fix a second, is. So vehicles now send every 1 s while moving, every 3 s in slow traffic and every 10 s when parked (emergency vehicles always every second), and the GPS itself drops to one fix every 10 s after half a minute parked, which is where the battery saving comes from. The GPS rate changes only between parked and driving, because restarting the background service is the app's most fragile path; a failed restart puts the old rate back. 13 tests.

What is deliberately not here

The source document is clear about the limits of the approach, and they belong in the plan as much as the features do:

- **Non-SUTRA vehicles, pedestrians and lanes.** That needs cameras and sensors on the vehicle. SUTRA is cooperative awareness between connected vehicles — a different mechanism, which scales with adoption rather than with hardware.
- **Control of real traffic signals.** Not legally available to a private application in India; it needs a city's adaptive signal control and police authority. Phase 12 documents the join rather than pretending it exists.
- **Live traffic data.** Routing uses speed limits, not congestion. Worth stating whenever "fastest route" is shown.

How each phase finishes

```
cd supabase/tests && npm test     # schema and RLS, Postgres in WASM
cd mobile && npm test && npm run typecheck
cd mobile && npx expo export      # catches missing peers before an EAS queue
```

Then: apply the migration and rehearse the live path inside a rolled-back transaction, deploy any changed function, redeploy the console, and rebuild the app only at the end of a group of phases that changed it.

Standing rule: every phase also updates `docs/project-record.html` and the README's feature, data-model and test-count tables. The record is the document an outside reader is given.
